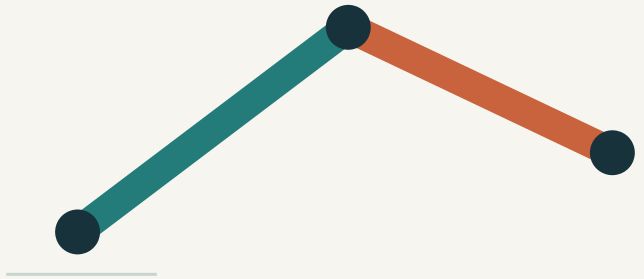
03

自分で動かす シミュレーションと MuJoCo

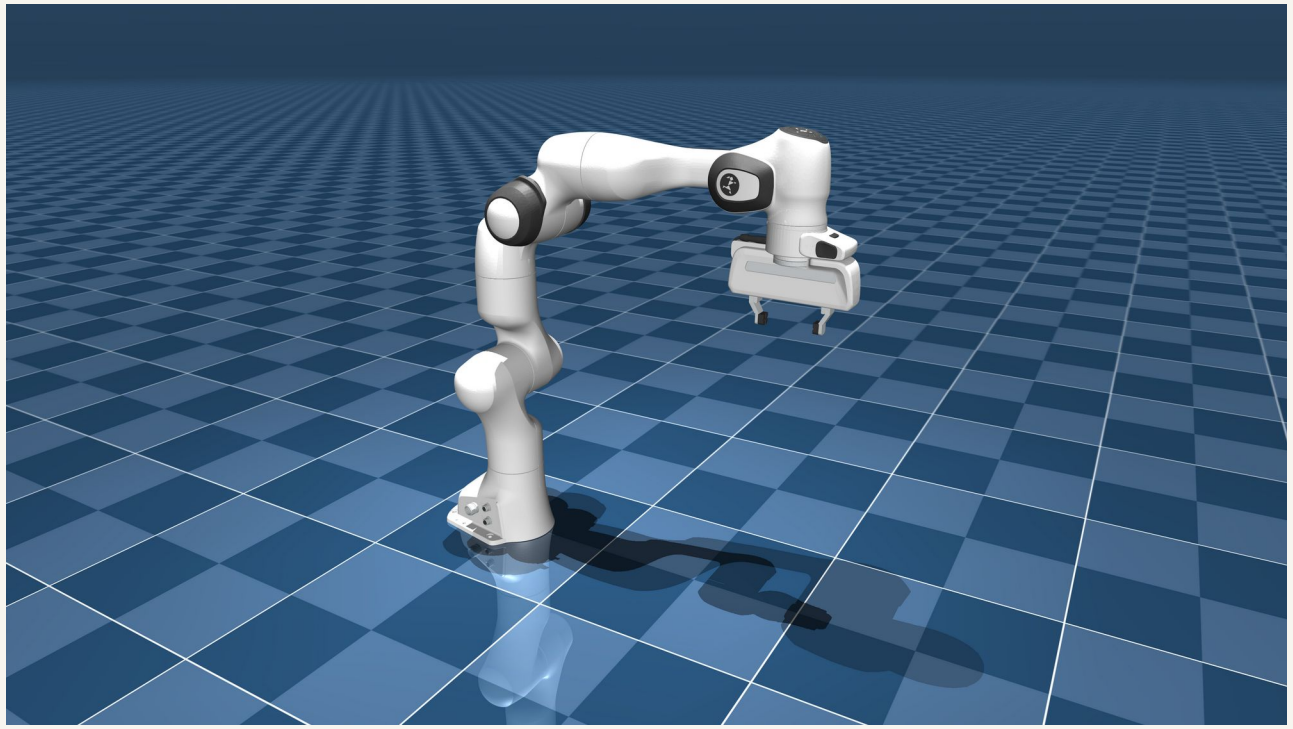


小球を動かす・条件を変える・結果を説明する

180 分 / 反転学習 / 必修 A ・ C ＋選択課題

シミュレータを学ぶ理由

実物を動かす前に、条件をそろえて試し、結果を比較する。



ロボットアームの仮想モデル

画像： MuJoCo Menagerie / Franka Emika Panda （シミュレーション画面）

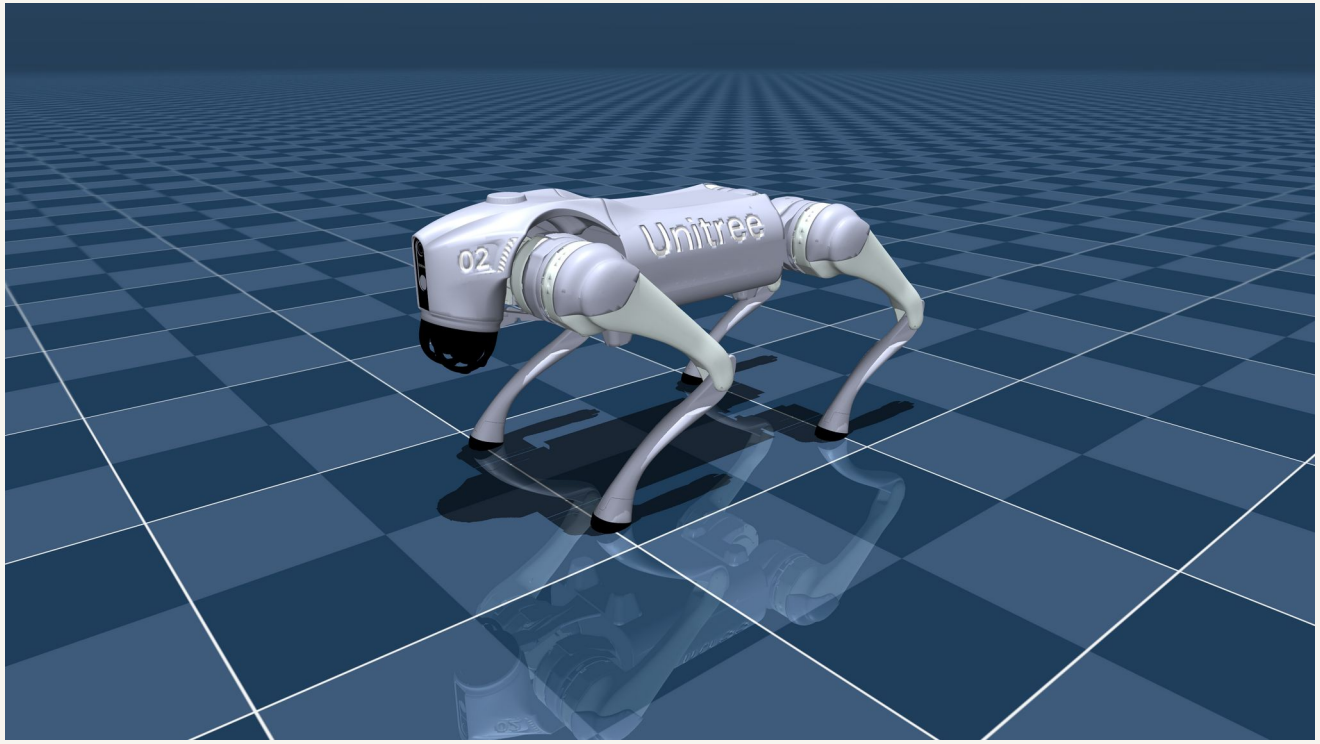
01 関節の角度を変える

02 質量・摩擦・入力を変える

03 同じ初期条件から比較する

ロボットの Sim-to-Real

仮想環境で調べた制御を、実機へ移して確かめる。



画像： MuJoCo Menagerie / Unitree Go2 （仮想モデル）

01 仮想モデルで制御を試す

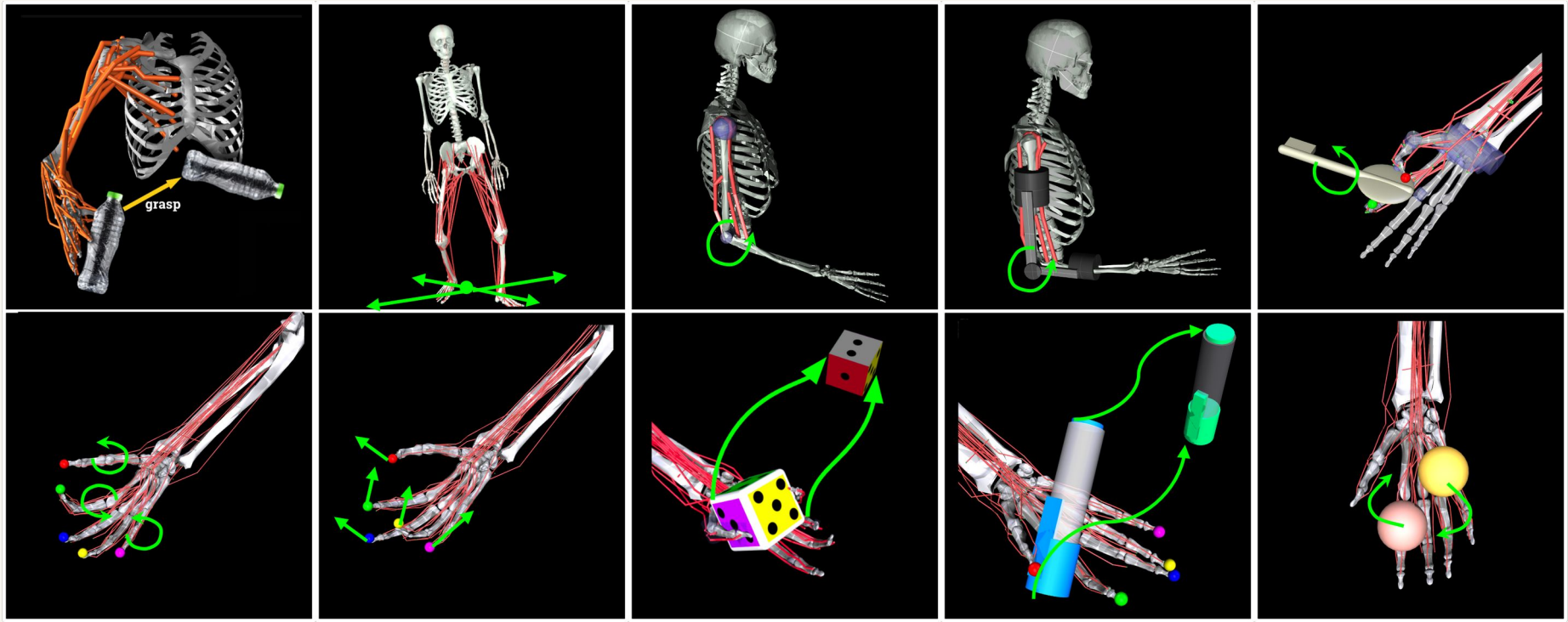
02 質量・摩擦・遅れを変える

03 実機で計測し、差を調べる

モデルと実物のずれが、
移した後の動きに影響する。

人体の運動をモデルで考える

骨・関節・筋のモデルから、動作や関節への負荷を調べる。



歩く・肘を曲げる・つかむ動作を、仮想の身体で計算する。

画像： MyoSuite 公式のモデル・課題例。今日の 2 リンクは、その仕組みを学ぶ簡略模型。

180 分の学習計画

必修は A ・ C 。予想を書き、一つずつ変更して確かめる。

時間 [分]	活動
0–30	第 2 回までの成果発表
30–40	仮想実験の画像と用途
40–60	小球の高さを変えて確認
60–85	必修 A： 速度と着地点
85–95	休憩
95–105	Euler の 2 行・時間刻み
105–140	MuJoCo の構成図・必修 C
140–150	リンク・関節の体験
150–160	Terminal で一度再現
160–180	相互確認・保存・発表準備

今日の必修は 2 つ

A： 速度と着地点を比較
C： 床の接触を修正

B ・ D ・ RK4 の詳しい式は選択。
リンクと Terminal は短い復習。
回答と保存を先に終える。
発表は 1 組 3 分＋質疑 2 分。

最初の成功： Python だけの落球

MuJoCo の準備より先に、自分で動く小球を見る。

```
dt = 0.02
z, v = 1.0, 0.0
for step in range(1, 41):
    z_next = z + v * dt
    v_next = v - 9.81 * dt
    z, v = z_next, v_next
```

01 z： 高さ [m]

02 v： 上向き速度 [m/s]

03 dt： 時間刻み [s]

Notebook の再生ボタンで確認。次に高さ 1.0 m を 1.5 m へ変える

物理計算とアニメーション

数値の列を計算し、その位置を順に画面へ描く。

シミュレーション

時刻ごとに
位置・速度を更新する。

地面を越えたら停止する
入門用のルールを使う。

アニメーション

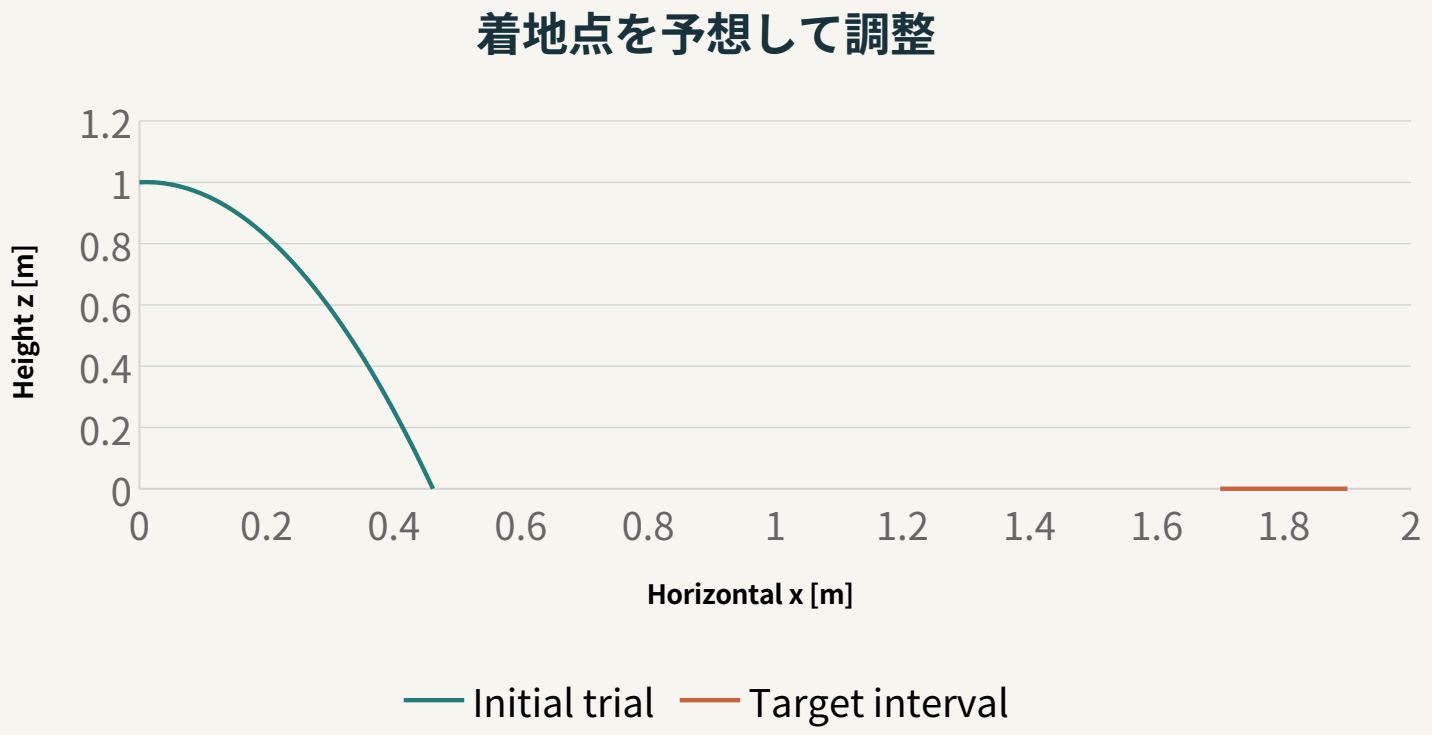
記録した位置を
順番に表示する。

再生速度を変えても
計算した軌道は変わらない。

仮想時間 1 秒と、画面で再生する 1 秒は同じでなくてもよい

必修 A： 目標区間へ小球を落とす

変えてよいのは、横向き初期速度 launch_vx だけ。



目標と条件

高さ： 1.0 m
横向き速度： 0 〜 6 m/s

目標： x=1.70 〜 1.90 m
重力・刻みは固定。

最低 2 条件を比べる。

顕式 Euler 法

最初は高さ・速度・ dt と、更新する 2 行が分かればよい。

$$\boldsymbol{s}_{n+1} = \boldsymbol{s}_n + \Delta t\, \boldsymbol{f}(\boldsymbol{s}_n)$$

```
z_next  = z  + vz * dt
vz_next = vz - g  * dt
z, vz = z_next, vz_next
```

記号とコードの対応

n： 現在のステップ
n+1： 次のステップ
Δt： コードの dt
s： 状態ベクトル

右辺は変更前の値。先に速度を書き換えると、別の更新方法になる

Euler の 2 行を数値で確かめる

配布例：高さ 1 m 、速度 0 、 dt=0.10 s 。導出は不要。

段階	高さ z [m]	速度 vz [m/s]
最初	1.0000	0.000
1 ステップ後	1.0000	−0.981
2 ステップ後	0.9019	−1.962

1 回目に位置が変わらない理由：変更前の速度が 0 だから。

MuJoCo が担当する計算

小球の手作り計算を、多関節・接触・入力へ広げる。

対象	必要になること
球だけ	力・加速度・時間積分
つながるリンク	質量・慣性・関節の制約
地面や物体との接触	接触位置・摩擦・反力
モータなどの入力	入力を力やトルクへ変換

MuJoCo： Multi-Joint dynamics with Contact

XML ・ MjModel ・ MjData の関係

装置の設定・計算用モデル・今の状態を、図で対応させる。

XML / MJCF

仮想の装置を書く
body ・ joint ・ geom
質量・重力・時間刻み

読み込み
コンパイル

MjModel

計算用のモデル
形状・物理パラメータ
関節や入力の構成

data を作成

MjData

位置・速度： qpos / qvel
時刻 time / 入力 ctrl
中間計算の値

```
model = mujoco.MjModel.from_xml_string(XML)
data = mujoco.MjData(model)
```

物理計算と描画は、この model と data を一緒に使う。 XML 変更後は読み込み直す。

最小の MJCF モデル

body ・ joint ・ geom を、一つの小球で読む。

```
<body name="ball">
 <joint type="slide"
       axis="0 0 1"/>
 <geom type="sphere"
      size="0.05"
      mass="1"/>
</body>
```

01 body： 剛体のまとまり

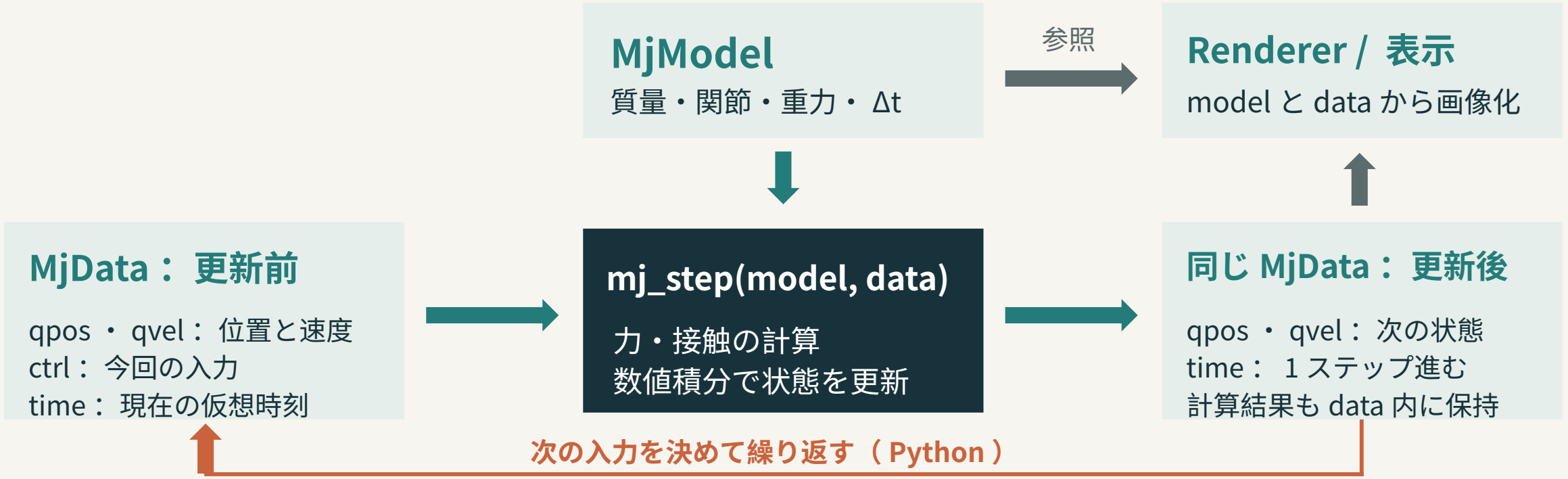
02 joint： 上下にだけ動ける

03 geom： 半径 0.05 m の形状

重力・刻み・積分法は option 、固定の床は worldbody に定義する

MuJoCo で 1 ステップ進める流れ

step が同じ data を更新する。描画と計算の役割を区別する。



表示・ CSV 保存は計算結果を読み取る。画像の表示だけでは時間は進まない。

mj_forward と mj_step

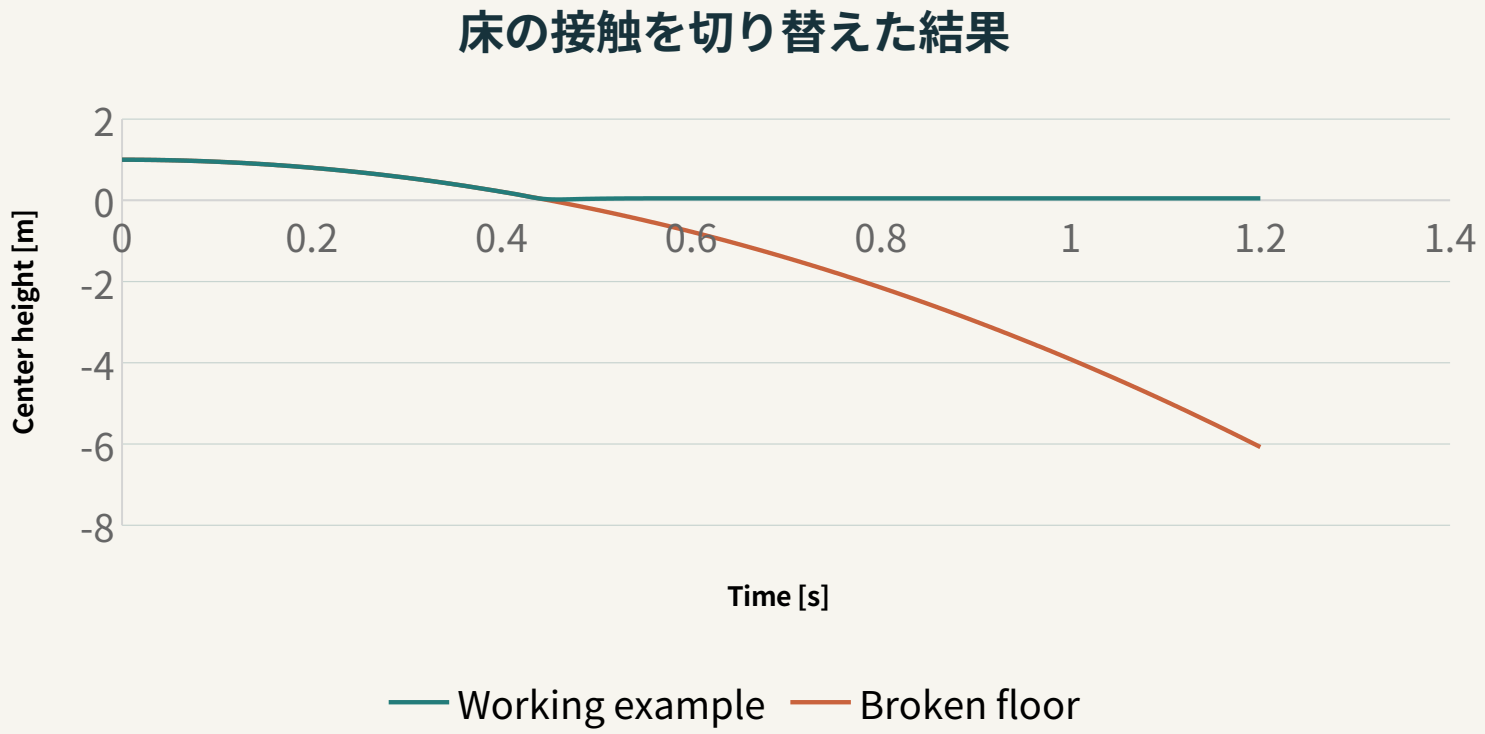
時刻・位置・速度を表示して、違いを自分で確認する。

命令	仕事	時間
qpos へ代入	関節位置・姿勢を設定	進まない
mj_forward	現在の状態で派生量を計算	進まない
mj_step	物理計算と積分で状態を更新	進む

forward を 10 回呼んでも落下しない。 step を 1 回呼ぶと変わる。

必修 C： 床の接触を有効にする

見た目の床があっても、接触が無効なら球は通り抜ける。



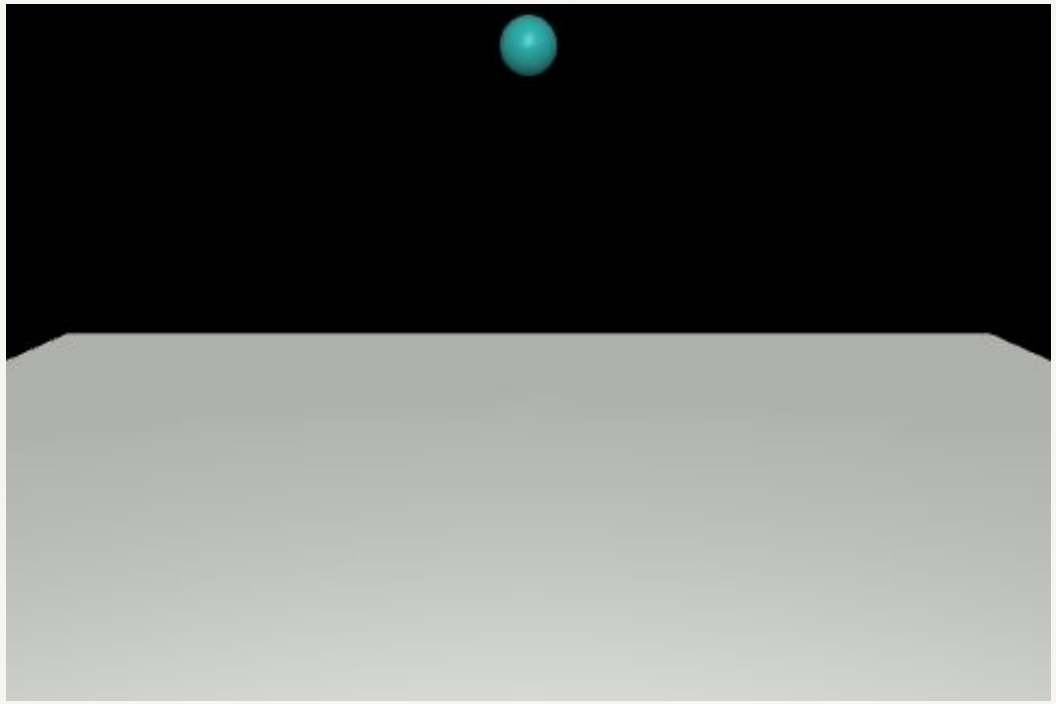
変えるのは一つ

floor_contact
False から True へ

最終の中心高さ
0.045 ～ 0.055 m
接触数も比較する。

Colab で見られるアニメーション

2D の小球も、 MuJoCo の 3D モデルも、セル内に表示できる。



01 CPU で運動を計算し、ログへ保存

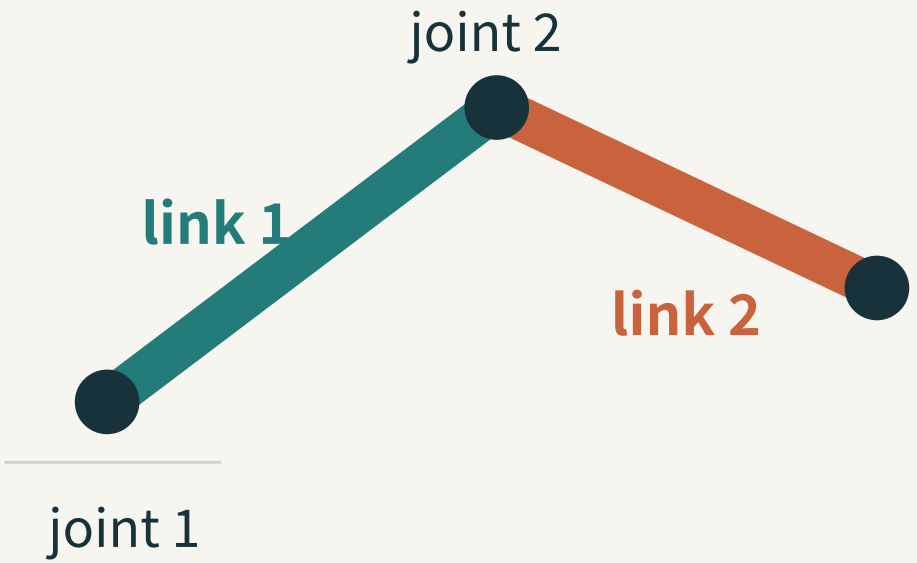
02 ログの状態を Renderer で画像化

03 HTML の再生ボタンで確認

描画セルで qpos を戻すのはログの再生。新しい力学計算ではない

リンクと関節の動きを見る

Notebook で肩の角度だけを 0° から 30° へ変える。



今日見るところ

link： 腕を作る部品
joint： 部品が動く場所

肩を回すと、肘の位置も
動くか確認する。
目標点の課題 D は選択。

Terminal で MuJoCo を実行する

配布済みのファイルを、一度実行して CSV を読む。

```
cd /content/b3_lesson03_v6
ls
python run_ball.py 1.0
head -n 3 terminal_ball.csv
```

余裕があれば

1.0 を 1.5 へ変更。
CSV の先頭行を読む。

cp で変更前を残す。

Terminal には
！ を付けない。

Terminal が表示されない環境は、 Notebook の代替セルか研究室 PC を使う

未達とプログラムのエラー

どこまで動いたかを確認し、変更する場所を絞る。

症状	最初に確認するもの
小球が動かない	更新式・ dt ・初期値・反復
XML Error	タグ・属性・ファイル名
関節が動かない	joint ・入力・ mj_step
3D 表示だけ出ない	CSV の有無・ Renderer ・描画環境
課題が未達	目標・出力・変更できる条件

AI へ質問するときも、目的・変更行・出力・予想との差を添える

変更と結果をセットで保存する

値を変えただけで終わらず、比較できる記録を残す。

01 trial_history.jsonl： 試した条件と判定

02 XML ・ Python ・ CSV： モデル、手順、結果

03 結果 ZIP： ランタイムが消える前に保存

04 回答入り Notebook： 自分用コピーを別に保存

Notebook をコピーしても、ランタイム内のファイルは自動で一緒にコピーされない

次回発表は、 A か C を選んで 3 枚

変更前後の結果と、値を変えた理由を説明する。

枚数	内容	示す根拠
1	問い・予想・固定条件	変更した行と値
2	変更前後の 2 条件の比較	図・数値・単位
3	結果の解釈・残った疑問	どの出力から考えたか

相手が同じ条件で再実行できるか、ペアで確かめる

ここまでで今日の必修は終了

動作・変更・比較・説明ができたか、ペアで確かめる。

01 小球を動かし、高さを変えて結果を確認した

02 A の速度・着地点、 C の接触設定・高さを比較した

03 XML ・ model ・ data ・ step の役割を図で説明した

04 回答入り Notebook と結果 ZIP を保存した

以降は選択資料。必要な部分だけ読む。公式資料は Notebook 末尾にある。

力・加速度・状態の関係

物理モデルが変化率を決め、数値積分が時間を進める。

$$F_z = -mg$$

$$a_z = \frac{F_z}{m} = -g$$

上向きを正、 g を重力加速度の大きさとする。
空気抵抗のない、接触前の落球。

状態ベクトルと変化率

$$\boldsymbol{s} = \begin{bmatrix} z \\ v_z \end{bmatrix}, \qquad \dot{\boldsymbol{s}} = \begin{bmatrix} v_z \\ -g \end{bmatrix}$$

高さと速度をまとめて扱う。
derivative が変化率を返す。

物理モデルの選択と、積分アルゴリズムの選択を分けて考える

古典的 4 次 Runge–Kutta 法： RK4

1 ステップの途中も調べ、 4 回の変化率を重み付きで使う。

$$\boldsymbol{k}_1 = \boldsymbol{f}(\boldsymbol{s}_n)$$

$$\boldsymbol{k}_2 = \boldsymbol{f}\left(\boldsymbol{s}_n + \frac{\Delta t}{2}\boldsymbol{k}_1\right)$$

$$\boldsymbol{k}_3 = \boldsymbol{f}\left(\boldsymbol{s}_n + \frac{\Delta t}{2}\boldsymbol{k}_2\right)$$

$$\boldsymbol{k}_4 = \boldsymbol{f}(\boldsymbol{s}_n + \Delta t\,\boldsymbol{k}_3)$$

$$\boldsymbol{s}_{n+1} = \boldsymbol{s}_n + \frac{\Delta t}{6}\left(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4\right)$$

各段階の意味

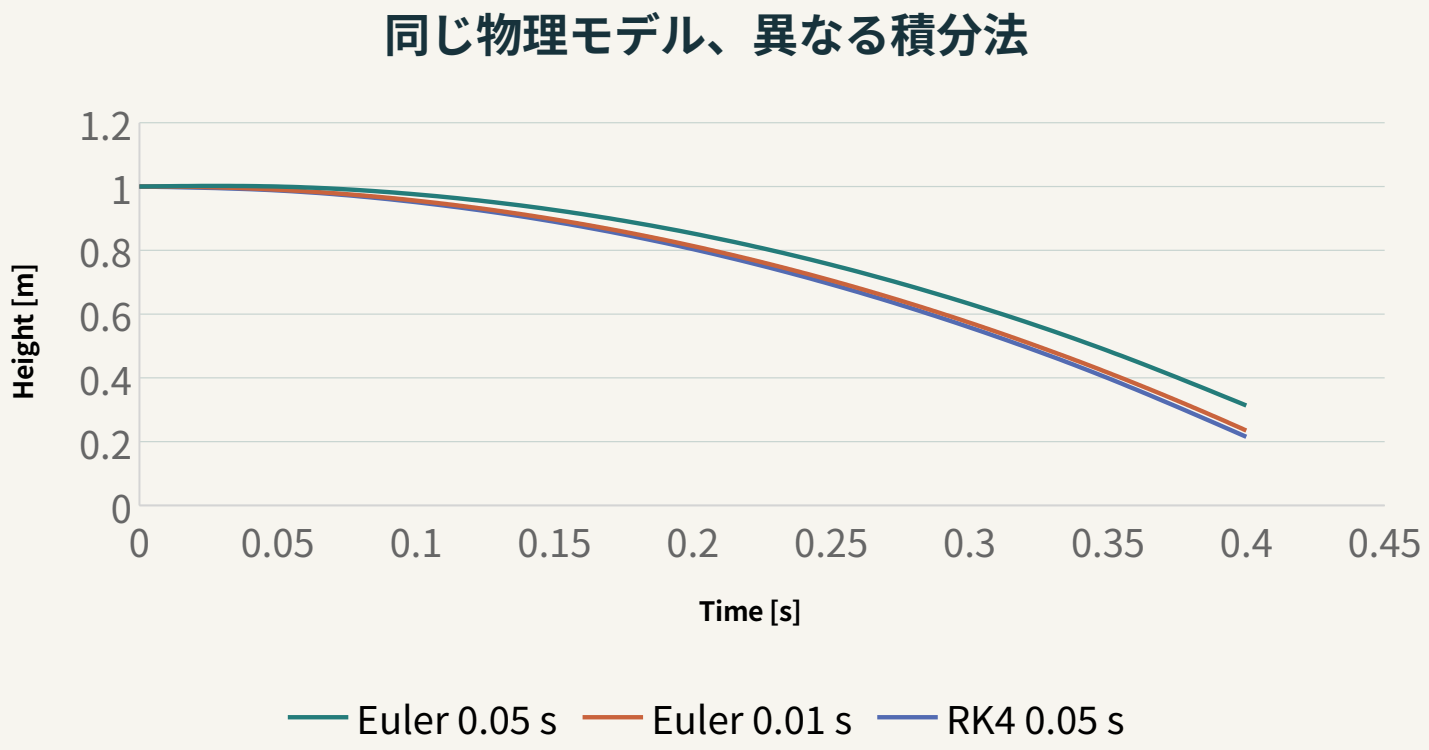
k_1： 現在の変化率
k_2 ・ k_3： 途中の予測
k_4： 終点の予測

Δt： コードの dt

進む時間は Δt 。 4Δt ではない。式の各段階を Notebook のコードで確かめる

同じ時刻で Euler と RK4 を比較する

初期高さ 1 m 、初期速度 0 、重力一定。接触前の 0.40 秒まで。



比較の基準：理論解

$$z(t) = z_0 + v_{z,0}t - \frac{1}{2}gt^2$$

初期高さ： 1 m
初期速度： 0 m/s

0.40 秒後は 0.2152 m 。
誤差も数値で比較する。

選択 B： 高さの誤差を 5 mm 以下にする

Euler のまま刻みを変え、次に RK4 でも比較する。

方法	Δt [s]	回数	高さの誤差 [m]
Euler	0.05	8	0.09810
Euler	0.01	40	0.01962
RK4	0.05	8	丸め誤差程度

$$e_z = \left| z_{\mathrm{num}}(T) - z_{\mathrm{exact}}(T) \right| \leq 0.005\,\mathrm{m}$$

T=0.40 s 。刻みの候補： 0.05 、 0.02 、 0.01 、 0.005 、 0.002 s

数値誤差とモデルのずれ

計算を細かくすることと、現実をよく表すことは別の改善。

数値誤差

同じ方程式を
どれくらい正確に解いたか。

刻みや積分法を変える。
理論解や収束で調べる。

モデルのずれ

省いた現象や
パラメータの違いは何か。

摩擦・空気抵抗・遅れなど。
実測と照合して調べる。

RK4 で正確に解けても、省略した物理現象は自動で追加されない

MuJoCo の Euler は更新順序が違う

同じ「 Euler 」という名前でも、実装の定義を確認する。

手書きの顕式 Euler

$$z_{n+1} = z_n + v_{z,n}\,\Delta t$$
$$v_{z,n+1} = v_{z,n} - g\,\Delta t$$

位置には、変更前の速度を使う。

MuJoCo の Euler 系

$$v_{z,n+1} = v_{z,n} - g\,\Delta t$$
$$z_{n+1} = z_n + v_{z,n+1}\,\Delta t$$

この無減衰・接触前の例では、
速度を先に更新して使う。

MuJoCo には RK4 ・ implicitfast などもある。方法と刻みをセットで考える

関節が動ける方向を決める

球形の geom と、 ball joint は別のもの。

関節	動き	例
hinge	1 軸の回転	ロボットの回転関節
slide	1 方向の移動	直動台・台車
ball	3 方向の回転	理想化した球関節
freejoint	並進 3 ＋回転 3	空間を自由に動く物体

freejoint は qpos が 7 成分、 qvel が 6 成分。配列の長さは常に同じではない

親子関係で 2 リンクをつなぐ

第 2 リンクの取り付け位置は、第 1 リンクを基準に記述する。

```
worldbody
  └ base： 固定
    └ link1： 肩で回転
      └ link2： 肘で回転
        └ tip： 末端の目印
```

このモデルの寸法

link1： 0.5 m
link2： 0.4 m

link2 の pos は
0.5 0 0

親 link1 を基準にした
取り付け位置。

2 リンクの XML を読む

体のつながり、動く軸、形と長さを順に追う。

```
<body name="link1">
  <joint name="shoulder"
         type="hinge" axis="0 0 1"/>
  <geom fromto="0 0 0 0.5 0 0" .../>
  <body name="link2" pos="0.5 0 0">
    <joint name="elbow" .../>
    <geom fromto="0 0 0 0.4 0 0" .../>
    <site name="tip" pos="0.4 0 0"/>
  </body>
</body>
```

... はスライドで省略した部分。実行する完全な XML は Notebook 内にある。

関節角度と末端位置

第 2 リンクの向きは、第 1 関節角と第 2 関節の相対角度の和。

$$\boldsymbol{q} = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}, \qquad \dot{\boldsymbol{q}} = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}$$

$$\theta_2 = q_1 + q_2$$

$$q_{\mathrm{rad}} = q_{\mathrm{deg}} \frac{\pi}{180}$$

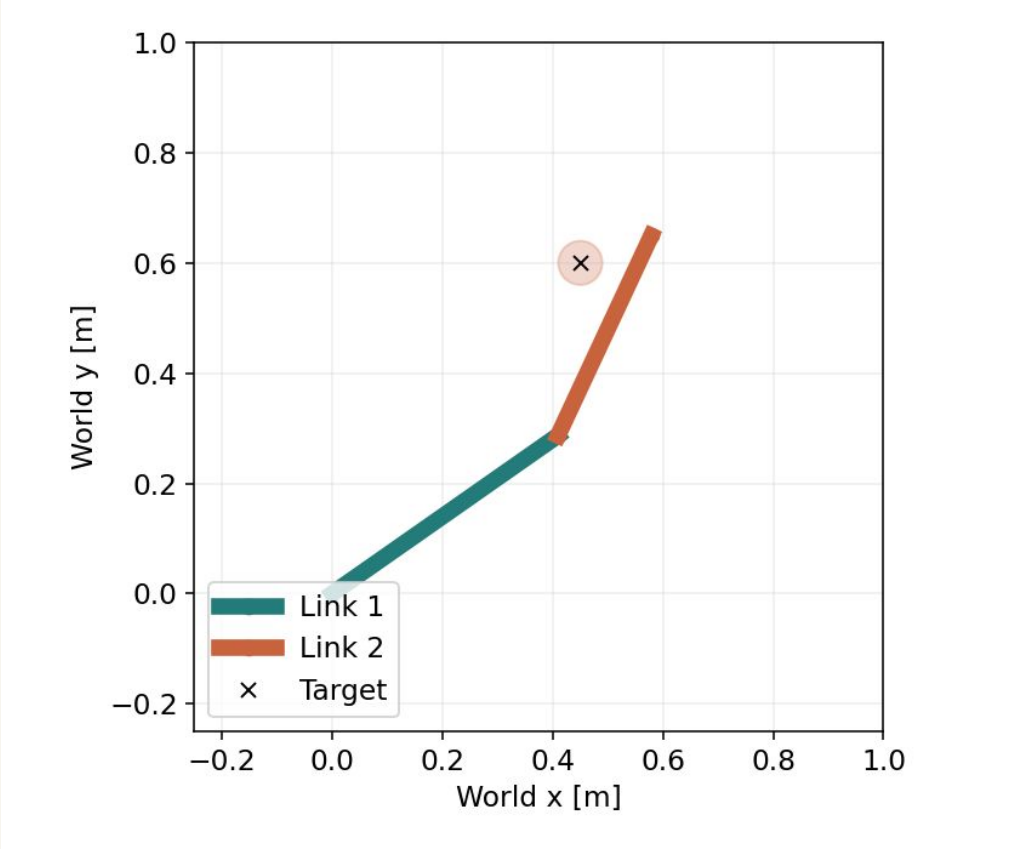
qpos：関節角度 [rad]
qvel：関節角速度 [rad/s]

コードでは np.deg2rad で変換。
末端位置は site("tip").xpos 。

qpos に 30 を入れると 30 rad 。 30° を指定するときは、先に rad へ変換する

選択 D： 手先を目標点へ近づける

まず q1 だけ、次に q2 だけを変え、どのリンクが動くかを見る。



達成条件

目標： (0.45, 0.60) m
誤差： 0.04 m 以内

q1_deg と q2_deg を調整。
リンク長と目標は固定。

最低 2 組の角度を記録する。

姿勢設定と動力学を区別する

自分で決めたものと、計算で求めたものを説明する。

選択 D： 運動学

自分で決める：関節角度
計算する：末端位置

qpos に代入
mj_forward

時間は進まない。

次の実験：動力学

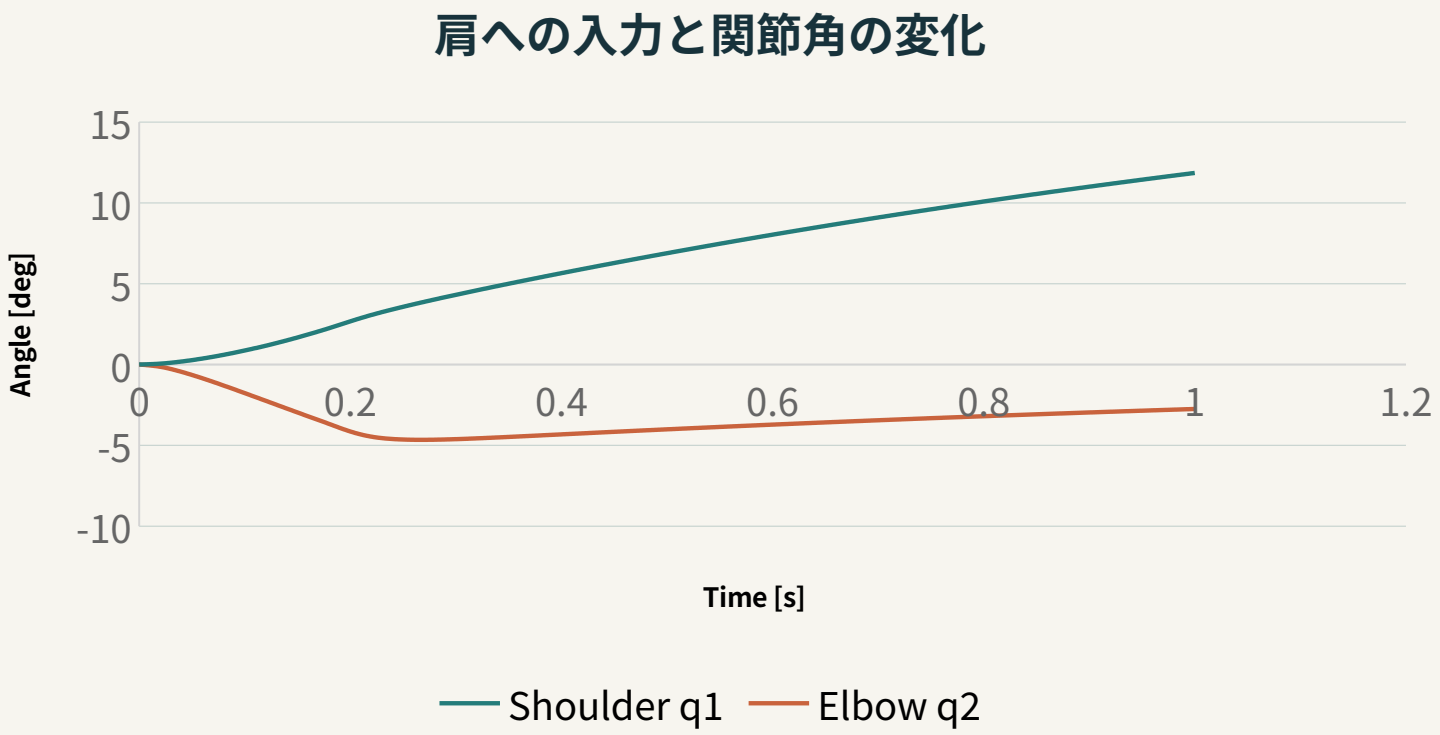
自分で決める：入力トルク
計算する：角度と速度の変化

ctrl に代入
mj_step

時間が進む。

トルクを与えた 2 リンクの運動

最初の 0.20 秒だけ肩に 0.20 N·m を与え、その後は入力 0 。



コードで確認

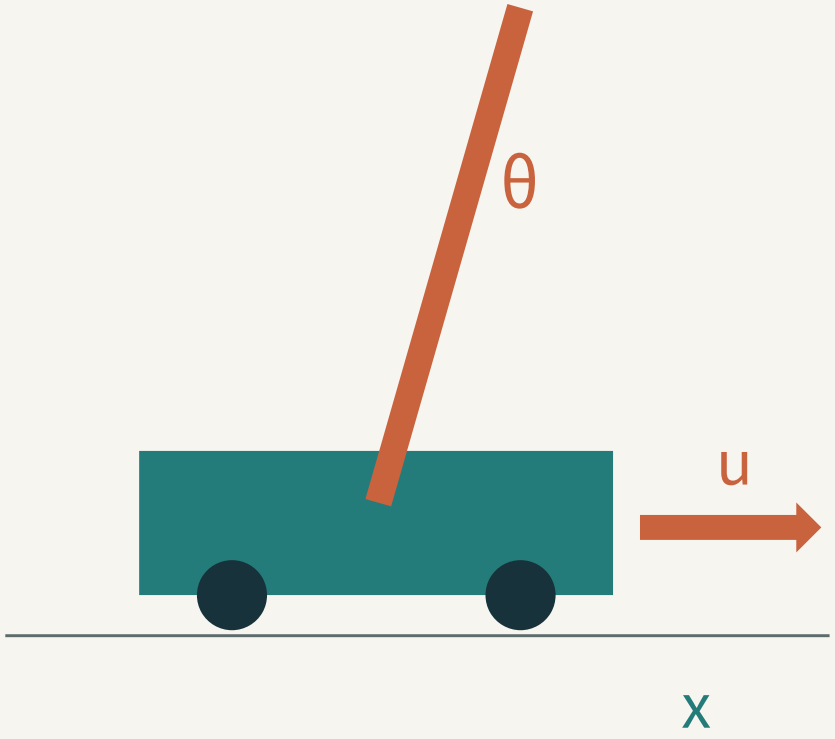
ctrl： 関節への指令
step： 状態を進める

hinge ＋ gear=1 の motor 。
この例の入力単位は N·m 。

肘への直接入力は 0 。

倒立振子も、リンクと関節で読める

台車の移動と棒の回転を組み合わせる。



01 cart： slide joint で左右に移動

02 pole： hinge joint で回転

03 motor： 台車へ力を加える

次の制御学習では、状態を読み、入力を繰り返し決める

発展：空気抵抗のある小球

同じ刻みでも、 Euler と RK4 の誤差はどう変わるか。

$$\frac{\mathrm{d}z}{\mathrm{d}t} = v_z, \qquad \frac{\mathrm{d}v_z}{\mathrm{d}t} = -g - cv_z$$

条件

$c = 2\ \mathrm{s}^{-1}$
$\Delta t = 0.05\ \mathrm{s}$
終了時刻 $= 0.40\ \mathrm{s}$

接触前の高さを比較する。

確かめること

理論解との差を数値で読む。
RK4 でも誤差は残るか。

運動方程式の違いと
計算方法の違いを区別する。